Reading guide

The move score head, and why it is built this way

Companion to `Chapters/4.Implementation.tex` (the move score)
and the L3 section of `fix_prs_plan.md`

This guide explains one design choice, how the decentralised policy turns what a robot sees into a choice between its exits. It adds no new model. The choice is already in the Method chapter as the move score (`eq:edgescore`) and is logged as `[A17]` in `thesis-progress/temp/GAPS.md`. The code change that brings `slap-mapd-coupling` into line with it is called L3 (issue #92).

Part I is the short version in everyday words. Part II explains the idea in more detail. Part III gives the reasoning, the alternatives that were weighed, the arguments for the choice and the arguments against it. Nothing here has been measured, and the places where that matters are listed in the last sections.

Contents

I The short version 2

1 The idea in plain words 2

2 Words used here 3

II The idea in more detail 3

3 The problem 3

4 What the current head sees 3

5 The fix 4

6 What has to be built 4

7 Formally 5

III The reasoning 5

8 What the thesis needs from a head 5

9 The options 6

10 Why not an LSTM 6

11 Van Knippenberg and PRIMAL 6

12 Arguments for the per-edge head 7

13 Arguments against, and weak points 7

14 Decision 14, the extra trunk 8

15 What is not verified 8

16 What the change does 8

Part I
The short version

1 The idea in plain words

A robot standing at a junction has to pick one of the roads leaving it, or wait. This guide is about how the policy makes that pick.

How it works today. The policy gets one description of the robot and its surroundings. It also has a row of numbered buttons, one for each road, plus a button for waiting. It must press one button. The buttons are numbered by the vertex number of the road they lead to, which is only a label. The description never says which button leads where.

Picture a driver in a country they do not know. At every junction a sign says "press 1, 2 or 3" and says nothing about where each button leads. After many trips the driver may learn that at one particular junction, button 3 is usually good. But at the next junction button 3 leads somewhere else, so what was learned is no use there.

The fix. Look at each road on its own. For every road, ask three things. Does it get me closer to my target? Is it blocked? Did the robot standing on it tell me anything? The answers go into one small marking rule, which gives the road a mark. The same rule marks every road, and the robot picks among the marks. Waiting gets its own mark.

Why that is better. The rule "a road that gets closer and is free is good" is true at every junction. So the robot learns it once and uses it everywhere, whatever the numbers are.

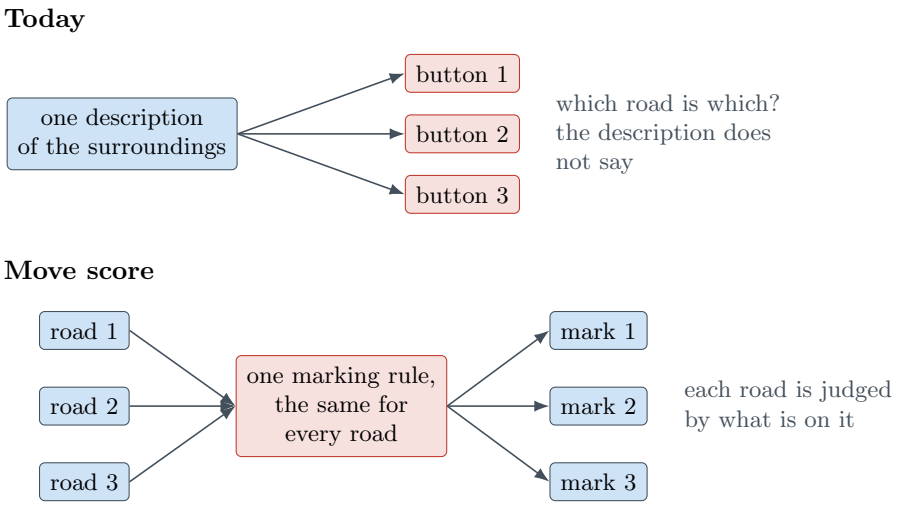


Figure 1: Today one description feeds numbered buttons. The move score marks each road by looking at it.

What it costs. The code must remember which road is behind which button, and which robot stands on which road. The network must also keep what it already works out about each road, instead of throwing it away. That bookkeeping is most of the work in L3.

What could go wrong. A road may lie outside what the robot can see, and then there is nothing to mark. A robot standing on a road may be too far away to be heard. Both cases need a rule. Section 13 lists them.

Why not something fancier. A memory unit such as an LSTM answers "what happened earlier", not "which road is which". It would also give a robot information from outside its window, which would spoil the comparison between the three controllers. Section 10 has the details.

2 Words used here

Word	Plain meaning
exit, road	A way out of the robot's current vertex, that is, a move it may take.
slot, button	The numbered place for an exit. Slot 0 is wait. Exits are numbered by vertex number.
summary (z_i)	One list of numbers describing the robot and its surroundings.
embedding	A list of numbers the network builds to describe one vertex.
message	The embedding another robot reports about its own surroundings.
score, mark	A number the network gives a move. A higher number makes the move more likely.
action mask	A rule that removes the moves that do not exist at this vertex.

Part II
The idea in more detail

3 The problem

A robot's action is a slot number. Slot 0 means wait. Slot k means take the k-th exit of the current vertex, counting the exits by increasing vertex number. The slot number says nothing about what lies behind the exit.

Figure 2 shows two junctions with three exits each. At junction A the exits lead to vertices 3, 8 and 12, so they are slots 1, 2 and 3. The target lies beyond vertex 12, so the right move is slot 3. At junction B the exits lead to vertices 5, 21 and 30, and the target lies beyond vertex 5, so the right move is slot 1.

A head that must answer slot 3 at one junction and slot 1 at the other, from nearly the same input, has to tell the junctions apart some other way.

4 What the current head sees

The current head turns one summary of the robot, z_i, into one number per slot. The summary holds the robot's own embedding, the average of the messages it received and the crowding $\delta_i(t)$. It holds nothing that says which exit is which.

Without message passing ($L = 0$) the own embedding is only the features of the robot's own vertex. The exits do not appear in it at all. With message passing the exits do appear, but only

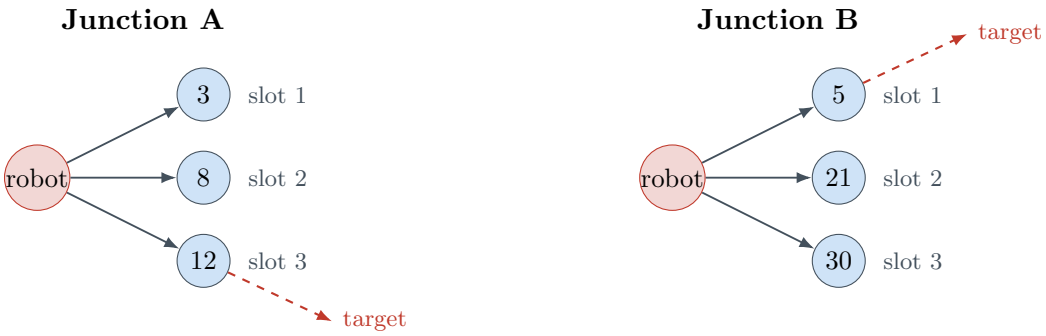


Figure 2: Two junctions. The right slot is 3 at A and 1 at B.

as an average, because the message-passing step combines neighbours as an unordered set. An average cannot say that the closer exit is slot 3.

What the head can learn is a habit tied to the numbering, such as "at this kind of vertex, slot 3 is usually good". That habit does not carry over to another junction, and it may break when another warehouse is numbered differently.

Key idea. The slot number is bookkeeping. The only thing that says what a move does is the vertex it leads to, so the head should look at that vertex.

5 The fix

Score each exit by what is behind it. For one exit, a small network is shown three things.

- The robot's own summary z_i.
- The embedding of the vertex behind the exit. It says how close that vertex is to the target, what role it has and whether it is occupied.
- The message of the robot standing on that vertex, or zeros if nobody is there.

It returns one number. The same network scores every exit at every junction, so what it learns is "prefer an exit that gets closer to the target and is not blocked". That lesson holds at junction A and at junction B alike. Waiting gets its own number from z_i. The numbers are placed into the slots in vertex order, and the action mask removes the slots that do not exist.

Table 1 shows the idea with made-up numbers. At junction A the robot is 4 away from the target, and at junction B it is also 4 away. One rule picks slot 3 at A and slot 1 at B.

Table 1: Made-up numbers for the junctions of Figure 2. Distance is to the target.

Junction	Slot	Vertex	Distance	Occupied
A	1	3	5	no
A	2	8	5	yes
A	3	12	3	no
B	1	5	3	no
B	2	21	5	no
B	3	30	6	yes

6 What has to be built

Three things were missing for this to work.

Piece	Why it is needed
The embedding behind each exit	The encoder already computes it for every vertex in the field of view and keeps only the robot's own. The change keeps all of them.
A slot table	The action is a slot number, so the head must know which embedding belongs to which slot. The environment records this in the observation, together with the robot standing on each exit.
A rule for exits it cannot score	An exit outside the field of view, or a robot that is not among the senders heard, leaves the head with no input. The code checks the first when the environment is built and gives a zero message for the second.

7 Formally

For an agent at $v = \ell_i(t)$ and each outgoing neighbour $w \in N^+(v)$, the move score is

$$\text{logit}(\text{move to } w) = \text{SCORE}_\theta\big(z_i \,\|\, h_w^{(L)} \,\|\, \bar{\xi}_w(t)\big), \qquad \text{logit}(\text{wait}) = \text{SCORE}_\theta^{\text{wait}}(z_i), \tag{1}$$

where $h_w^{(L)}$ is the final-layer embedding at w and $\bar{\xi}_w(t)$ is the message of the agent standing on w, zero if there is none. With communication off, $\bar{\xi}_w(t)$ is dropped. The policy is the softmax of these logits plus the action mask $M(v, \cdot)$, as before.

Part III
The reasoning

8 What the thesis needs from a head

The thesis compares three learned controllers that differ only in what they may see. So the head should be a neutral part that is the same for all three. It should work on warehouses generated from seeds, and it should be simple enough to finish in the time left. Judged by that, a head that cannot tell exits apart is the one real defect. Everything else is taste.

9 The options

Option	What it does	Verdict
Fixed head as today	One output per slot from z_i.	Cannot tell exits apart. At $L = 0$, z_i holds no neighbour information.
Fixed head plus vertex ID	Van Knippenberg's way.	The network learns numbers, not geometry. It does not carry over to another numbering.
Slot concatenation	Feed z_i with the embeddings of exits 1 to $d_{\max}$ side by side.	Simple, and each exit sees the others. But the weights grow with $d_{\max}$ and each slot learns the same lesson separately.
Sort exits by distance to the target	Slot 1 is always the exit closest to the target.	A fixed head then works. But it builds the shortest-path planner into the policy and contradicts `[A15]`.
Score every vertex in view	The action is "which vertex".	No slot table, but the action space is no longer small and shared, and it contradicts `[A15]`.
Dot-product attention	Score is a dot product, not a network.	Fewer weights, but it learns fewer interactions. The thesis says "one small fully connected network", and attention is future work (`[A18]`).
Per-edge head	One shared network scores each exit.	Same lesson at every junction, any number of exits, weights independent of $d_{\max}$.

10 Why not an LSTM

An LSTM does not make the head cheaper and does not fix the problem. There are three ways to use one.

As memory across timesteps. This gives the robot a hidden state. It answers a different question, "what happened earlier", and does not say which exit is which, because that is a spatial question at a single timestep. It also works against the thesis. The three controllers differ only in their window (`tab:information`), and the critic reads the same window as its policy (`[A13]`). A hidden state carries information from outside that window, so a difference between the controllers could come from memory and not from the window.

It is also costly to build. The state would have to be carried through episodes by RLlib. The learned message recomputes each sender's embedding with the current weights, so each sender's state would need recomputing as well.

As a sequence over the exits. Pointer networks were first written with an LSTM. But an LSTM reads the exits in order, so the result depends on the numbering again, which is what the move score removes. It also costs more than a small network per exit, because it has four gates and runs one step after another.

Inside the message passing. A recurrent unit in place of ψ_r is a known design. It changes the encoder and not the head, and it adds weights.

11 Van Knippenberg and PRIMAL

Read in the paper (`knippenberg21a.pdf`). The action space is "a small action space ... shared among all agents, and across problem instances", defined as local movement options, one move across each outgoing edge plus wait. The model is a graph convolution, then fully connected

layers with ReLU, then a softmax over one fixed output vector. Every node carries a node ID. That ID is the only thing in the input that could tell two outgoing edges apart, and the paper does not say how move k is matched to an edge. An invalid move is treated as a penalised wait.

PRIMAL (`1809.03531v3.pdf`) ends its network in an LSTM, on lattice graphs only. Van Knippenberg dropped the LSTM, and the thesis has none.

The move score keeps van Knippenberg's principle. The action set is the same small shared set, $d_{\max} + 1$ slots, and the head is, if anything, more shared than theirs. What changes is how the logits are computed. The thesis already says so in its action-space paragraph, "this is where the head departs from van Knippenberg". An earlier decision (`[A16]`) kept their head and noted the node-ID caveat. `[A17]` replaced it.

12 Arguments for the per-edge head

1. **It removes a real defect.** The current head cannot tell exits apart, and the reason is visible in what z_i holds.
2. **One lesson, learned once.** The shared network learns "prefer an exit that gets closer and is free" from every junction the robots visit, and not once per slot.
3. **Numbering does not matter.** Relabelling the slots only permutes the logits. This is a direct test of the thesis claim that the vertex numbering no longer affects what the policy learns.
4. **It is neutral between the controllers.** The centralised and section-based controllers read all their robots' vertices from one shared pass, so the same head fits them. No controller gets a special head, which keeps the window comparison clean.
5. **A message sits at the exit it concerns.** The message of the robot on an exit is an input to that exit's score. This is what lets communication help the choice of a move, and it is the reason `[A17]` gave.
6. **Any number of exits.** The weights do not depend on $d_{\max}$.
7. **Small cost.** The head runs a few small networks per robot. The encoder is the expensive part, and the learned message already multiplied it by one plus the number of senders. This is from counting operations. It has not been profiled.
8. **Little RLlib machinery.** The change is what the actor network reads. It needs no connector, unlike the message option that was rejected in L2b.
9. **The thesis already commits to it.** The code catches up with the text, and not the other way round.

13 Arguments against, and weak points

1. **Each exit is scored alone.** The head cannot say "both exits are blocked, so wait" except through the softmax and z_i. Slot concatenation could model that directly.
2. **The plumbing has a risk.** RLlib has to pass a head more than one tensor. In the RLlib source the training forward pass hands the actor's embedding straight to the policy head, so a dictionary should work. The inference and value paths were not checked in the source. The build starts with a spike for this, and falls back to one flat tensor if it fails.
3. **Exits can lie outside the field of view.** The field of view uses cost distance, the depth d_{obs} is an integer, and the generator can draw random edge costs between 0.5 and 2.0. Depth

1 appears in five places in the tests and examples. The code will check at construction that every exit is inside the view, and this may reject some existing configurations. It has not been run.

4. **An unheard sender gives a zero message.** The number of messages is capped, and most test and example configurations use 1 or 2. A junction with three occupied exits can then have an exit without a message. The thesis says "if there is one" and does not cover the cap.

5. **It adds a choice to the padding-caps decision.** The view must keep every exit, and two checks follow from that. This touches the open decision on padding caps (decision 15).

6. **Nothing was measured.** The claim that the fixed head cannot learn the lesson rests on what z_i holds, not on a run.

7. **Numbering may carry direction.** If the generator numbers vertices systematically, slots may behave like directions, and the fixed head could be good enough on that family. I have not tested this.

8. **Compute is not an argument.** The head is cheap either way, so a cheaper head would save almost nothing. A simpler head saves engineering effort, and slot concatenation needs nearly the same changes.

Key idea. If the thesis wants evidence for the choice, run the current fixed head on one small instance before L3 lands and keep the number. Then a reviewer's question "why per-edge?" has a measured answer. This depends on the training budget (decision 2).

14 Decision 14, the extra trunk

Today a shared hidden layer, the trunk, sits between z_i and both the policy and the value output. With the move score the policy no longer reads a trunk, it reads the three inputs of Section 5. A trunk shared with the value would then serve only the value and the wait score.

The author decided on 2026-10-13 to drop it (option a), so the move score, the wait score and the value are three small networks that each read their own input. This matches the thesis, where the value head is "a fully connected network with ReLU activations over z_i". It also closes the gap item "the head adds its own trunk" by making the code match the text.

The text needs no edit, but the figure does. `fig:architecture` draws one "fully connected + ReLU" box between the readout and both heads, which is the trunk. Under (a) that box goes and the readout feeds the three networks. The figure edit is still to be made.

15 What is not verified

- No part of this has been trained or timed. The claims about what the fixed head can learn come from reading the code.
- The RLlib inference path for a head with several inputs.
- Whether existing test and example configurations pass the new field-of-view check.
- How often the zero message for an unheard sender occurs.

16 What the change does

The code change, L3, is built on its own branch and is run and pushed by the author. It makes three changes, in this order.

1. The observation gets a slot table, and the field of view always keeps every exit.
2. The encoder hands over the embedding of every vertex, not only the robot's own.
3. The head becomes the move score, the wait score and the value, and the RLlib module is wired to it.

   The full list of files, tests and open choices is in the L3 section of `fix_prs_plan.md`.